

Roteamento de modelos e orquestração de agentes para engenharia de software: qualidade, custo e latência

Isaac

Resumo—Taxas de resolução, isoladamente, não descrevem os recursos necessários para que agentes de programação concluam tarefas nem permitem atribuir diferenças ao modelo quando harnesses e ferramentas variam. Este artigo discute evidências sobre custo, efeitos de scaffold, exposição a benchmarks e amostragem eficiente e apresenta um protocolo para avaliar roteamento entre modelos em tarefas de engenharia de software. O protocolo compara configurações sob tarefas, ferramentas e limites documentados, mede qualidade, custo e latência separadamente e distingue triagem de avaliação confirmatória. O uso seletivo de um modelo de maior capacidade para planejamento ou escalonamento, combinado com execução de menor custo e possível paralelização, permanece uma hipótese de projeto. Ainda não foram obtidos resultados experimentais que permitam afirmar que essa arquitetura preserve qualidade ou reduza recursos.

Palavras-chave—agentes de programação, roteamento de modelos, engenharia de software, scaffolding, avaliação de benchmarks

I. INTRODUÇÃO

Em agentes de programação, a taxa de resolução informa se uma tarefa foi concluída segundo o verificador, mas, sozinha, não informa quanto foi necessário gastar para obtê-la. Repetições, chamadas adicionais e revisão podem alterar simultaneamente a probabilidade de sucesso e o consumo de recursos. Kapoor et al. identificam a ênfase exclusiva em acurácia como uma limitação da avaliação de agentes e formulam a otimização conjunta de acurácia e custo; seus experimentos mostram que considerar custo pode mudar a interpretação de estratégias que parecem superiores quando se observa apenas a taxa de sucesso [1]. Para decisões de engenharia, portanto, a comparação precisa mostrar a qualidade alcançada junto com os recursos consumidos, em vez de tratar qualquer ganho de resolução como vantagem suficiente.

A necessidade de avaliar qualidade e recursos conjuntamente torna o harness parte da condição experimental. Em um estudo controlado com 50 tarefas do Terminal-Bench Pro e os modelos Qwen 3.6 Plus e MiniMax M2.5, as diferenças pareadas de taxa de resolução entre harnesses variaram de 0 a 8 pontos percentuais. Os intervalos de confiança incluíram zero na maioria das comparações, o que não demonstra equivalência entre os harnesses. Ao se considerar o consumo de tokens por tarefa resolvida no mesmo estudo, observaram-se diferenças de até 40 vezes. Esse contraste reforça que a taxa de resolução, isoladamente, omite diferenças substanciais de consumo mesmo em um contexto experimental controlado [2]. Como a avaliação abrangeu apenas dois modelos e três

harnesses, o fator de 40 vezes demonstra a possibilidade de grande variação, mas não estima a magnitude do efeito em outros modelos, sobretudo modelos mais recentes ou mais capazes. Tokens tampouco equivalem a um custo monetário fixo, pois tarifas, cache e categorias de cobrança variam.

Além de afetar custo e desempenho, a infraestrutura pode determinar se uma alteração produzida pelo agente chega sequer a ser testada. O Claw-SWE-Bench evidencia esse problema na interface entre agente e avaliador. Em sua configuração mínima, o modelo precisava escrever um *diff* unificado na resposta final; erros em caminhos, cabeçalhos ou linhas de contexto podiam impedir a aplicação do patch antes da execução dos testes. Na configuração completa, o agente editava os arquivos do repositório, e a camada de integração calculava o *diff* em relação ao estado inicial, removia artefatos alheios à solução e produzia o patch no formato aceito pelo avaliador. Com o mesmo modelo GLM 5.1 e as mesmas 350 tarefas, a taxa de resolução passou de 19,1% para 73,4%, enquanto a proporção de patches não aplicáveis caiu de 69,1% para menos de 1,5% [3]. O contraste não atribui esse ganho apenas à extração do patch, pois a configuração completa também modificou outros elementos do adaptador. Ele demonstra, contudo, que a pontuação pode confundir a capacidade de resolver a tarefa com a capacidade de satisfazer o contrato de entrega. A unidade de comparação deve, assim, identificar modelo, harness, ferramentas, orçamento e procedimento de entrega.

Mesmo uma comparação controlada pode ser inválida se as tarefas avaliadas não medirem capacidade de resolver problemas novos. No SWE-bench Verified, um estudo pediu a dois modelos Claude que localizassem arquivos relevantes usando o texto da issue, com contexto deliberadamente limitado, e comparou esse desempenho ao de BeetleBox e SWE-rebench. Os modelos tiveram desempenho três vezes maior no Verified e seis vezes maior na identificação de arquivos editados sem contexto adicional; os autores interpretam a diferença como compatível com exposição prévia às tarefas, mas o experimento não revela diretamente os dados de treinamento nem prova memorização de soluções específicas [4]. Para a validade da avaliação, isso sustenta registrar a idade e a origem das tarefas, separar desenvolvimento de avaliação e restringir a interpretação de resultados em benchmarks públicos; não autoriza atribuir automaticamente diferenças entre conjuntos à contaminação.

O custo de avaliar muitas combinações de modelos, harness-

ses e benchmarks também condiciona o desenho do estudo. Ndzomga examina a redução do conjunto de tarefas por dificuldade histórica e encontra uma faixa intermediária útil para preservar a ordenação de agentes em vários cenários, enquanto a estimativa de pontuações absolutas se mostra mais sensível, especialmente quando muda o scaffold [5]. Esse resultado fundamenta uma etapa econômica de triagem, mas não transforma uma amostra escolhida para preservar rankings em estimativa confirmatória da taxa de resolução no benchmark inteiro. O protocolo deste estudo separa essas finalidades e reserva a avaliação de qualidade para tarefas que não tenham sido usadas para ajustar ou selecionar configurações.

A validade de um benchmark envolve ainda a correspondência entre o que ele pontua e o trabalho que se espera de um agente de engenharia de software. Gorinova et al. argumentam que benchmarks de programação podem confundir o efeito do modelo com o do harness, vincular a correção à solução de referência e fornecer pouco sinal sobre a origem das falhas [6]. Por se tratar de um artigo de posição, essas críticas não quantificam a prevalência de cada problema nos conjuntos usados aqui. Elas apontam, porém, limites de interpretação que a avaliação pode enfrentar com controles de componentes, registro de falhas e inspeção independente de uma amostra de entregas, mantendo o veredito oficial do benchmark como medida operacional principal.

Resultados fora do domínio de programação reforçam que o efeito de um scaffold pode depender do modelo e da tarefa. No estudo controlado de GAIA, os efeitos variaram entre famílias de modelos e níveis de dificuldade; a expectativa de que o scaffold se tornaria sistematicamente menos relevante com modelos mais capazes não se confirmou no nível mais difícil avaliado [7]. Como esse benchmark mede tarefas gerais com ferramentas, e não resolução de issues de software, o resultado não antecipa a direção do efeito em engenharia de software. Ele mantém em aberto, como hipótese exploratória, se a capacidade do modelo modera o benefício do scaffold e justifica medir essa interação em vez de pressupor uma tendência uniforme.

Em conjunto, essas evidências sustentam a tese metodológica deste artigo: a avaliação de agentes de programação deve comparar qualidade sob limites explícitos de custo e tempo, controlar o harness como parte do sistema e tratar a validade das tarefas como condição para interpretar o resultado. A arquitetura em estudo explora uma hipótese ainda não confirmada: distribuir planejamento, execução, escalonamento e, quando viável, subtarefas paralelas entre modelos de custos distintos pode melhorar essa relação. As seções seguintes especificam essa configuração e um protocolo para confrontá-la com agentes de modelo único e controles de repetição ou escalonamento, sem antecipar resultados experimentais.

II. SCAFFOLD PROPOSTO

O sistema em estudo será composto por um orquestrador, agentes com papéis definidos e mecanismos de verificação. A configuração inicial prevê que um modelo mais capaz produza um plano ou especificações delimitadas para as tarefas; um

modelo de menor custo executará essas tarefas usando ferramentas de navegação, edição e teste. O orquestrador acompanhará dependências, resultados intermediários, orçamento e critérios de encerramento.

Essa distribuição de papéis é uma hipótese de projeto, não uma conclusão da literatura examinada: o planejamento por um modelo de maior capacidade pode orientar a execução de menor custo, mas também pode acrescentar chamadas e falhas de coordenação. Permanece igualmente em aberto se a contribuição relativa do scaffold diminui à medida que cresce a capacidade do modelo. O estudo de GAIA não encontrou uma redução monotônica no nível mais difícil e observou diferenças entre famílias; como suas tarefas e modelos não correspondem à avaliação de código aqui proposta, esse resultado motiva manter a hipótese exploratória e medir a interação, sem antecipar sua direção [7].

Quando uma subtarefa não puder ser resolvida com as evidências disponíveis, o sistema poderá encaminhar o caso ao modelo mais capaz para diagnóstico, revisão ou orientação adicional. Os gatilhos e limites desse escalonamento ainda serão definidos. Para que o efeito do roteamento possa ser medido, cada intervenção deverá ser registrada, incluindo seu motivo, conteúdo, custo e efeito sobre a solução.

A paralelização será considerada apenas para subtarefas que possam ser executadas sem dependências conflitantes. Os resultados dos agentes paralelos precisarão ser integrados e verificados. A hipótese é que essa estratégia possa reduzir o tempo decorrido; seu efeito líquido sobre custo e qualidade será medido, pois a coordenação e a integração também consomem recursos.

A decisão de que o agente terminou não será usada como medida suficiente de correção. Durante a execução, o controlador poderá considerar testes disponíveis, critérios de aceitação e evidências registradas. A avaliação final das tarefas será realizada pelo protocolo externo do benchmark. Quando forem usados revisores baseados em modelo, eles não substituirão essa avaliação externa, e sua possível discordância com o executor será registrada.

Os componentes serão implementados de forma configurável para permitir comparações e ablações. A configuração mínima de controle será um agente de modelo único com as ferramentas necessárias à tarefa, mas sem o roteamento ou a coordenação multiagente propostos. Essa definição operacional evita confundir a contribuição do scaffold com uma comparação entre um agente equipado e um modelo sem acesso ao repositório ou às ferramentas.

III. METODOLOGIA PROPOSTA

A. Desenho comparativo

Será conduzida uma avaliação comparativa por tarefa. Cada configuração receberá a mesma entrada, o mesmo estado inicial do ambiente e ferramentas equivalentes. A ordem das execuções será intercalada ou aleatorizada quando viável, e as tarefas usadas para desenvolver o arcabouço serão separadas das reservadas à avaliação final.

Para esta pesquisa, “modelo puro” designará o modelo executando como agente único em um harness básico, com acesso às ferramentas indispensáveis à tarefa, mas sem o roteamento e a orquestração propostos. Uma chamada de chat sem acesso ao ambiente não constitui baseline equivalente para tarefas de edição de repositório ou de terminal.

A unidade de comparação será a configuração completa: identificador do modelo, versão do scaffold e do adaptador, ferramentas, prompts, limites de tempo e gasto e procedimento de entrega. Em uma comparação entre modelos, o harness será fixado; em uma comparação entre scaffolds, o modelo será fixado. A interface de avaliação, o estado inicial do repositório e a extração do patch serão comuns às condições.

Nos benchmarks de resolução de issues, o agente editará uma cópia de trabalho iniciada no commit definido pela instância; não dependerá da redação manual de um *diff* em sua resposta final. Depois do encerramento da execução, uma camada comum do protocolo calculará as diferenças em relação ao estado inicial, removerá apenas artefatos previamente especificados e produzirá o patch no formato requerido pelo avaliador. O mesmo coletor, as mesmas regras de limpeza e a mesma versão do avaliador serão usados em todas as condições. Serão preservados o estado final da cópia de trabalho e o patch submetido para permitir auditoria. Essa separação entre edição e entrega segue a evidência de que pedir ao modelo que serialize diretamente um patch pode introduzir falhas de aplicação anteriores à execução dos testes [3].

O contrato de entrega será validado antes das comparações principais. Ausência de alterações, erro na extração, rejeição do patch pelo avaliador e reprovação após sua aplicação serão registrados como etapas distintas. Todas contarão como insucesso na taxa de resolução quando decorrerem da configuração avaliada, mas sua apresentação separada evitará interpretar incompatibilidade de formato como evidência direta de incapacidade para resolver a issue.

B. Seleção e justificativa dos benchmarks

O SWE-bench consolidou a avaliação de agentes que recebem uma issue e um repositório e devem produzir um patch verificável por execução de testes [8]. O SWE-bench-Live será o conjunto principal para esse tipo de tarefa: sua coleção atualizável reúne issues mais recentes, de um número maior de repositórios, com ambientes executáveis [9]. A atualidade reduz uma fonte potencial de exposição prévia das tarefas, mas não comprova ausência de contaminação. Tampouco uma diferença de pontuação entre conjuntos demonstra, isoladamente, memorização: a dificuldade e a composição das tarefas também podem variar.

O Multi-SWE-bench complementarà a avaliação por incluir 1.632 tarefas verificadas em Java, TypeScript, JavaScript, Go, Rust, C e C++ [10]. O estudo desse conjunto relata diferenças de desempenho entre linguagens e entre métodos de agente. O SWE-PolyBench também documenta desempenho desigual e distribuição não uniforme de tarefas por linguagem, reforçando a necessidade de apresentar resultados desagregados [11].

Python aparece nas comparações publicadas do Multi-SWE-bench por meio de tarefas de outro conjunto; não será contado como uma das sete linguagens que compõem suas 1.632 instâncias.

O Terminal-Bench será a terceira avaliação, voltada à execução de tarefas por agentes em ambientes de linha de comando. Sua publicação acadêmica apresenta o Terminal-Bench 2.0 e um protocolo de verificação por tarefa [12]. O projeto mantém versões posteriores; a versão 4.0 alterou tarefas, verificadores e recursos de execução [13]. A versão e o identificador do conjunto serão fixados antes dos experimentos. Como suas tarefas abrangem também domínios além da engenharia de software, seus resultados serão interpretados como evidência complementar sobre atuação em terminal, e não como taxa de resolução de issues.

A inclusão de cada instância exigirá ambiente reproduzível, instruções suficientes e um verificador aplicável. Auditorias do SWE-bench Verified e do SWE-bench Pro encontraram, respectivamente, problemas de exposição de soluções e de adequação dos testes, e problemas de qualidade em tarefas do conjunto Pro [14], [15]. Esses achados motivam a auditoria e o registro de problemas nas instâncias usadas; não serão extrapolados automaticamente para o SWE-bench-Live, o Multi-SWE-bench ou o Terminal-Bench. Não haverá agregação das pontuações dos três benchmarks em um escore único.

Há evidência adicional para examinar contaminação de maneira específica. Prathifkumar et al. solicitaram a dois modelos a localização dos arquivos alterados em issues do SWE-bench Verified, inclusive com apenas o texto da issue, e observaram desempenho substancialmente maior nesse conjunto do que em BeetleBox e divisões mais recentes do SWE-rebench [4]. A diferença é compatível com exposição prévia, mas não identifica, por si só, o conteúdo do treinamento nem prova que uma solução particular foi memorizada. Por isso, a avaliação final deverá registrar datas das tarefas e versões dos modelos, separar instâncias usadas no desenvolvimento e evitar acesso às soluções de referência; resultados entre conjuntos diferentes não serão tratados como teste causal de contaminação.

C. Amostragem e orçamento de avaliação

O custo de uma grade completa de benchmarks, modelos, scaffolds e ablações pode ultrapassar o orçamento disponível. Após um piloto para estimar custo e variabilidade por tarefa e condição, serão fixados o teto total de gasto, os limites por tentativa e as configurações que cabem na avaliação. Cada comparação usará as mesmas tarefas nas condições confrontadas. O plano distinguirá triagem de configurações e avaliação confirmatória, reservando tarefas não usadas para ajustar o scaffold ou selecionar políticas de roteamento.

Para triagem, poderá ser empregado um subconjunto informativo selecionado por taxas históricas de sucesso por tarefa, quando esses dados estiverem disponíveis e a faixa de dificuldade intermediária tiver tamanho suficiente. Ndzomga observou que selecionar tarefas com taxa histórica entre 30% e 70% preservou bem a ordenação de agentes em vários benchmarks, reduzindo o número de tarefas, mas a previsão

de pontuações absolutas degradou sob mudança de scaffold [5]. A regra e sua origem serão fixadas antes de executar as novas configurações, sem usar seus resultados para escolher as próprias tarefas. Um subconjunto desse tipo servirá para decidir quais alternativas merecem avaliação adicional, não para apresentar sua taxa bruta de resolução como estimativa da taxa no benchmark completo.

Na etapa confirmatória, será priorizado um conjunto reservado com seleção probabilística previamente especificada, eventualmente estratificada por benchmark, linguagem, repositório ou classe de tarefa. Serão registrados universo elegível, probabilidades de inclusão, exclusões e pesos necessários para estimar a taxa correspondente ao universo definido. Se o orçamento impedir uma amostra com precisão suficiente para avaliar a margem de não inferioridade, a conclusão será limitada às tarefas executadas e não afirmará qualidade comparável no benchmark inteiro. Antes de congelar o protocolo, o piloto informará a análise de precisão do contraste pareado e a escolha entre ampliar a amostra, reduzir o número de condições ou restringir o escopo da conclusão. O Terminal-Bench poderá permanecer como avaliação complementar em um recorte pré-definido, sem receber alegações de desempenho global não sustentadas por sua amostra.

D. Condições experimentais

O desenho inicial contempla as seguintes condições:

- 1) agente de modelo único com o modelo de menor custo;
- 2) agente de modelo único com o modelo de maior capacidade;
- 3) cada modelo de referência executando com o arcabouço proposto, para estimar o efeito de acrescentá-lo ao mesmo modelo;
- 4) configuração com roteamento entre os modelos e alocação de papéis;
- 5) configuração roteada com execução paralela de subtarefas independentes, caso a decomposição seja aplicável.

As condições serão refinadas após um piloto e a leitura dos estudos sobre arquiteturas e roteamento. Para isolar efeitos, serão comparadas a configuração completa e ablações sem roteamento, sem revisão ou escalonamento e sem paralelização. Modelos, versões, prompts, limites, ferramentas, parâmetros de geração e orçamento serão registrados. Além de comparações sob orçamento fixado, será relatado o consumo efetivamente observado.

Serão incluídos controles simples sob o mesmo teto de recursos, como o agente básico com repetição limitada após falha observável e uma regra fixa de escalonamento ao modelo mais capaz. Esses controles são necessários para verificar se eventuais ganhos da arquitetura decorrem de sua política de coordenação ou apenas de novas tentativas e maior gasto de inferência [1]. O número de tentativas internas, o número de execuções experimentais independentes e o limite de passos serão definidos separadamente. A mesma política de acesso aos testes disponíveis ao agente valerá para todas as condições; testes reservados permanecerão externos.

E. Métricas de qualidade e conclusão

Para cada benchmark b , configuração c e tarefa i de um conjunto previamente fixado I_b , seja $s_{i,c} = 1$ quando o verificador oficial considerar a entrega correta, e $s_{i,c} = 0$ caso contrário. A medida primária de qualidade operacional será a taxa de resolução, calculada por

$$\hat{R}_{b,c} = \frac{1}{|I_b|} \sum_{i \in I_b} s_{i,c}. \quad (1)$$

No caso dos benchmarks de issues, um patch deve ser aplicável e satisfazer os testes *fail-to-pass* (correção) e *pass-to-pass* (ausência de regressão) exigidos pelo respectivo protocolo [8]. No Terminal-Bench será usado o verificador da versão selecionada. Falta de entrega, patch inválido e timeout contarão como insucesso na medida primária, com suas causas descritas separadamente. Instâncias consideradas inválidas em auditoria prévia somente poderão ser excluídas por critérios registrados antes da comparação e aplicados a todas as configurações.

Qualidade, custo e latência serão apresentados como desfechos centrais distintos. A análise de qualidade empregará a margem de não inferioridade previamente definida para a diferença pareada de resolução em relação ao agente de referência; somente após essa verificação se examinará se custo e tempo diminuam na mesma comparação. Também serão apresentados casos de dominância e de troca entre dimensões, sem condensá-las em um índice único. A taxa observada em um subconjunto escolhido pela dificuldade será rotulada como resultado daquele subconjunto.

A taxa de aplicação de patches e , quando disponível, a fração de testes *fail-to-pass* aprovados serão diagnósticos secundários. Uma aprovação parcial não será confundida com tarefa resolvida, distinção também adotada na análise do FeatureBench [16]. Com uma tentativa por tarefa, a taxa de resolução corresponde ao sucesso em uma tentativa; se houver repetições, serão relatadas separadamente, sem transformar a melhor tentativa em resultado de tentativa única.

A confiabilidade da decisão interna de término será medida pela proporção de entregas declaradas concluídas que falharem na avaliação externa:

$$\hat{F}_{b,c} = \frac{\sum_{i \in I_b} \mathbb{1}[d_{i,c} = 1 \wedge s_{i,c} = 0]}{\sum_{i \in I_b} \mathbb{1}[d_{i,c} = 1]}, \quad (2)$$

em que $d_{i,c} = 1$ indica que o sistema declarou a tarefa concluída. A medida ficará indefinida se não houver entregas declaradas concluídas. Recusas, escalonamentos e encerramentos sem entrega serão relatados à parte. Esse indicador é proposto para este estudo e não substitui a pontuação oficial de nenhum benchmark.

F. Custo e latência

O custo monetário de API por tentativa incluirá todas as chamadas feitas pelo sistema, inclusive planejamento, execução, revisão, resumos, repetições e tentativas malsucedidas. Para a chamada j , $u_{j,k}$ será o número de tokens na categoria de cobrança k (entrada não armazenada em cache, entrada em

cache ou saída) e $p_{m_j,k}$ seu preço por milhão de tokens para o modelo m_j . Com eventuais tarifas fixas f_j , o custo será

$$C_{i,c}^{\text{API}} = \sum_{j \in J_{i,c}} \left(\sum_k \frac{u_{j,k} p_{m_j,k}}{10^6} + f_j \right). \quad (3)$$

As categorias serão mutuamente exclusivas para evitar dupla contagem. Serão registrados moeda, fonte e data das tarifas, assim como uso de tokens e eventuais chamadas sem dados de cobrança. Custos de infraestrutura e ferramentas serão informados separadamente do custo de API. Relataremos o custo médio por tarefa, $\sum_{i \in I_b} C_{i,c}^{\text{API}} / |I_b|$, e o custo por tarefa resolvida, $\sum_{i \in I_b} C_{i,c}^{\text{API}} / \sum_{i \in I_b} s_{i,c}$; esta última razão ficará indefinida se nenhuma tarefa for resolvida. O custo médio por issue é também relatado no Multi-SWE-bench [10].

O numerador do custo por tarefa resolvida incluirá também tarefas malsucedidas; portanto, a medida representa o gasto amortizado necessário para obter uma resolução sob a configuração observada. Serão relatados, como diagnósticos, tokens de entrada e saída, cache, número de chamadas, tentativas, testes executados e intervenções do modelo mais capaz. Os tokens permitem reavaliar custos quando tarifas mudarem, mas não substituem o valor monetário: preço por categoria, cache e eventuais cobranças de ferramenta podem mudar a relação entre consumo de tokens e fatura. Quando disponível, o custo calculado a partir dos logs será confrontado com os registros de cobrança.

A latência operacional de ponta a ponta será $T_{i,c} = t_{i,c}^{\text{entrega}} - t_{i,c}^{\text{inicio}}$, desde o fornecimento da tarefa ao agente até sua entrega ou encerramento. Inclui espera por modelos, ações de ferramenta, testes internos, coordenação e trabalho paralelo; o tempo posterior de avaliação externa será registrado à parte. Serão informados mediana, percentil 95 e distribuição por tarefa. O tempo de parede permitirá avaliar o efeito da paralelização sem confundir-lo com a soma dos tempos de todos os agentes.

G. Procedimento e análise

As tarefas serão executadas sem expor aos agentes testes reservados ou soluções de referência do avaliador final. Serão preservados patches, logs, chamadas a modelos, ações de ferramenta, testes executados, decisões de escalonamento e motivos de encerramento. Falhas de ambiente, timeouts e erros do agente serão classificados separadamente.

Também serão registrados o ponto da falha e o componente envolvido: localização, edição, aplicação do patch, execução de testes, verificação, escalonamento e decisão de parar. Isso fornecerá sinais sobre os componentes além do veredito final, sem substituir a avaliação de ponta a ponta. A taxonomia será definida antes da análise principal e casos ambíguos serão auditados. Em uma amostra viável de patches aprovados e reprovados, testes comportamentais adicionais ou revisão independente poderão verificar requisitos não cobertos pelo avaliador oficial; os critérios dessa revisão deverão ser definidos sem consultar a identidade da configuração avaliada.

No Multi-SWE-bench, serão apresentadas taxas por linguagem e por repositório, além da taxa global; uma média

entre linguagens com pesos iguais poderá ser relatada como medida secundária, sempre acompanhada dos tamanhos de cada grupo. O tamanho da amostra e o número de execuções por tarefa serão definidos após um piloto. A análise respeitará o pareamento por tarefa e a dependência entre tarefas do mesmo repositório. Intervalos de confiança e uma margem de não inferioridade para qualidade serão definidos antes da avaliação confirmatória. Custo, qualidade e latência serão exibidos separadamente, inclusive por relações de dominância de Pareto quando apropriado. A ausência de diferença estatisticamente significativa, por si só, não será interpretada como equivalência.

IV. DISCUSSÃO E AMEAÇAS À VALIDADE

A seleção de benchmarks envolve uma tensão entre atualidade, diversidade e custo de execução. Um conjunto atualizado pode reduzir algumas formas de exposição a tarefas públicas, mas não permite declarar ausência de contaminação sem evidência específica. A cobertura multilíngue, por sua vez, não garante que tarefas de linguagens distintas sejam comparáveis em dificuldade, tamanho ou domínio.

O contraste de localização com contexto mínimo observado por Prathifkumar et al. no SWE-bench Verified reforça essa ameaça [4]. Contudo, desempenho superior em um conjunto mais antigo também pode refletir diferenças de distribuição e familiaridade geral com seus repositórios. A interpretação de um eventual contraste entre SWE-bench-Live, Multi-SWE-bench e Terminal-Bench exigirá levar em conta tarefa, linguagem, dificuldade, data e protocolo; uma queda de pontuação em tarefas recentes não será atribuída automaticamente a memorização. Separar desenvolvimento e avaliação por tarefa e, quando viável, por repositório reduz sobreajuste do scaffold às instâncias observadas, mas não demonstra que os modelos nunca tiveram acesso a elas.

Os testes de um benchmark são uma medida operacional de correção e podem não cobrir todos os requisitos de uma issue. Assim, a taxa de resolução deverá ser interpretada como desempenho segundo o protocolo adotado, não como garantia geral de qualidade, segurança ou manutenibilidade do código. A decisão interna do agente de encerrar também deve ser distinguida do veredito externo.

Gorinova et al. argumentam que benchmarks de programação apresentam três desalinhamentos com a engenharia de software agêntica: confundem componentes do sistema em uma pontuação única, podem vincular a correção à forma de uma solução de referência e oferecem pouco sinal sobre onde uma execução falhou [6]. Trata-se de um artigo de posição, cujas propostas não estabelecem a frequência desses problemas nos conjuntos aqui selecionados. Para esta pesquisa, a implicação operacional é combinar o veredito oficial com ablações de componentes, classificação de falhas e, em uma amostra delimitada, verificação comportamental independente. Uma solução alternativa que falhe em testes dependentes da implementação de referência e um patch que passe em testes insuficientes representam erros de medição distintos; ambos

deverão ser examinados antes de extrapolar a taxa de resolução para qualidade de software em uso.

Modelos e serviços podem mudar ao longo do tempo, e preços, limites de uso e latência podem variar por provedor. A reprodutibilidade exigirá registrar os identificadores e versões dos modelos, os parâmetros, as datas de acesso, os preços e eventuais falhas de serviço. A comparação entre scaffolds também pode ser confundida por diferenças de ferramentas, prompts, orçamento ou harness; esses fatores deverão ser controlados ou documentados.

O estudo controlado em GAIA encontrou efeitos de scaffold dependentes de modelo e dificuldade, inclusive sem a redução monotônica esperada para modelos mais capazes no nível difícil [7]. Nesse nível, a vantagem do scaffold com múltiplos papéis apareceu com maior clareza na família Anthropic, mas não nos modelos de outros provedores examinados. Os scaffolds estruturados também fizeram menos chamadas de ferramenta, terminaram mais rapidamente em média e recuperaram-se com maior frequência de erros intermediários; isso mostra que acrescentar etapas de organização não implica necessariamente mais ações ou maior gasto. Ainda assim, sua transferência para engenharia de software é limitada pelo domínio avaliado e pelo fato de o controle ReAct usar uma superfície de ferramentas diferente da dos outros scaffolds. Já Vats e Golev observaram grande variação em tokens por tarefa resolvida entre harnesses de programação, mas a amostra de 50 tarefas e diferenças nos controles de turnos limitam a generalização do fator relatado para custo monetário ou para os modelos futuros [2]. Portanto, resultados anteriores justificam medir a interação modelo–scaffold e o consumo efetivo, sem fixar sua magnitude no presente estudo.

A redução do conjunto de tarefas introduz outra ameaça. O filtro por dificuldade intermediária estudado por Ndzomga foi eficaz para preservar rankings de agentes em certos conjuntos, mas não assegurou calibração das pontuações absolutas sob mudança de scaffold; além disso, requer resultados históricos suficientes e pode falhar quando poucas tarefas ocupam a faixa selecionada [5]. Assim, um ranking obtido na triagem não será usado como evidência de não inferioridade no conjunto completo. A seleção final e sua incerteza deverão ser descritas em termos da população de tarefas que se pretende representar. Se o orçamento permitir apenas um recorte não probabilístico, as conclusões serão restritas a esse recorte.

Por fim, resultados em um conjunto, idioma ou par de modelos não serão generalizados automaticamente a outras linguagens, repositórios ou modelos. As fontes examinadas fundamentam a necessidade de medir qualidade, recursos e validade em conjunto, mas não estabelecem que o roteamento proposto seja superior a agentes de modelo único. Essa conclusão dependerá de uma comparação pareada sob orçamento documentado e de uma margem de qualidade definida previamente. Se os critérios de qualidade, custo e latência apontarem em direções distintas, o resultado deverá ser apresentado como uma troca entre dimensões, e não como superioridade geral.

V. CONCLUSÃO

As evidências examinadas sustentam que a taxa de resolução, isoladamente, não basta para comparar agentes de programação sob restrições de engenharia. O consumo de recursos pode variar entre estratégias com resultados próximos; o harness pode mudar tanto o custo quanto a possibilidade de entregar uma solução avaliável; e exposição das tarefas ou amostragem orientada apenas à preservação de rankings limita o alcance das conclusões. Esses achados justificam avaliar, em condições controladas, qualidade, custo e latência, além de documentar o conjunto de tarefas, o harness e o procedimento de entrega.

O protocolo proposto operacionaliza essas implicações por meio de comparações pareadas, registros de custo por chamada, controles de repetição e escalonamento e separação entre triagem e avaliação confirmatória. O roteamento de um modelo mais capaz para planejamento ou intervenção seletiva, com execução de menor custo e paralelização quando aplicável, permanece uma hipótese a ser testada. Como ainda não há resultados experimentais, este manuscrito não conclui que essa arquitetura mantenha qualidade ou reduza custo e tempo; tais afirmações dependerão da avaliação sob um orçamento e uma margem de qualidade definidos previamente.

REFERÊNCIAS

- [1] S. Kapoor, B. Stroebel, Z. S. Siegel, N. Nadgir, and A. Narayanan, “AI Agents That Matter,” arXiv:2407.01502, 2024. [Online]. Available: <https://arxiv.org/abs/2407.01502>
- [2] N. Vats and O. Golev, “The scaffold effect in coding agents: Harness choice as a hidden variable in coding-agent evaluation,” arXiv:2607.22585, 2026. [Online]. Available: <https://arxiv.org/abs/2607.22585>
- [3] M. Zheng, K. Han, B. Li, H. Xu, Y. Tian, W. He, H. Zhou, J. Guo, H. Hu, L. Ma, C. Xu, G. Dai, L. Xia, Y. Wei, Y. Wang, and Y. Wang, “Claw-SWE-Bench: A benchmark for evaluating OpenClaw-style agent harnesses on coding tasks,” arXiv:2606.12344, 2026. [Online]. Available: <https://arxiv.org/abs/2606.12344>
- [4] T. Prathifkumar, N. S. Mathews, and M. Nagappan, “Does SWE-Bench-Verified test agent ability or model memory?” arXiv:2512.10218, 2025. [Online]. Available: <https://arxiv.org/abs/2512.10218>
- [5] F. Ndzomga, “Efficient benchmarking of AI agents,” arXiv:2603.23749, 2026. [Online]. Available: <https://arxiv.org/abs/2603.23749>
- [6] M. I. Gorinova, M. Baker, A. Heineke, M. Shaposhnikov, R. Willoughby, and D. Knox, “Position: Coding benchmarks are misaligned with agentic software engineering,” arXiv:2606.17799, 2026. [Online]. Available: <https://arxiv.org/abs/2606.17799>
- [7] J. Starace, “Scaffold effects on GAIA: A controlled comparison,” arXiv:2606.08529, 2026. [Online]. Available: <https://arxiv.org/abs/2606.08529>
- [8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub issues?” in *International Conference on Learning Representations*, 2024.
- [9] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, “SWE-bench Goes Live!” arXiv:2505.23419, 2025.
- [10] ByteDance Seed, “Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving,” arXiv:2504.02605, 2025.
- [11] M. S. Rashid *et al.*, “SWE-PolyBench: A multi-language benchmark for repository level evaluation of coding agents,” arXiv:2504.08703, 2025.
- [12] M. A. Merrill *et al.*, “Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces,” arXiv:2601.11868, 2026. [Online]. Available: <https://arxiv.org/abs/2601.11868>
- [13] R. Marten, “Terminal-Bench 4.0: Calibrating task resources, fixing tasks, and removing saturated tasks,” Terminal-Bench blog, Aug. 2026. [Online]. Available: <https://www.tbench.ai/news/terminal-bench-4-0>

- [14] OpenAI, “Por que o SWE-bench Verified não mede mais capacidades de codificação de ponta?” Feb. 2026. [Online]. Available: <https://openai.com/pt-BR/index/why-we-no-longer-evaluate-swe-bench-verified/>
- [15] —, “Separando sinal de ruído em avaliações de programação,” Jul. 2026. [Online]. Available: <https://openai.com/pt-BR/index/separating-signal-from-noise-coding-evaluations/>
- [16] Q. Zhou *et al.*, “FeatureBench: Benchmarking agentic coding for complex feature development,” in *International Conference on Learning Representations*, 2026.